RXPIP

The REXX Package Manager

Introduction, Installation and User's Guide

Release 1 • updated 12 October 2026

for Regina, ooRexx, NetRexx and cREXX and the repository www.rexxpository.org

Publication RXP-0001-0
File No. REXX-01

Second Edition (October 2026)

This edition applies to RXPIP Release 1 (clients for Regina, ooRexx, NetRexx and cREXX, and the server, as of 12 October 2026) and to all subsequent releases until otherwise indicated in new editions.

RXPIP is a hobbyist project. It is not a product of, and this manual is not a publication of, any company named in it. REXX is the Restructured Extended Executor language. Regina REXX, Apache HTTP Server, AlmaLinux, Hetzner, Let’s Encrypt, curl and the other programs and services named here are the property of their respective owners and are named only to identify what RXPIP runs on and works with.

The project repository holds the source of the client and server, the examples and the automated test suite from which every statement in this manual about behaviour can be checked. Comments on this publication may be addressed to the project.

Contents

Preface 0

Chapter 1. Introduction 0

- The parts of RXPIP 0
- How a program uses a package 0
- Artifacts: source and bytecode 0
- Supported REXX interpreters 0

Chapter 2. Installing the Client 0

- What you need 0
- Linux and macOS 0
- Windows 0
- The other clients 0
- Pointing the client at a repository 0
- Files in the home directory 0

Chapter 3. Using Packages 0

- Finding packages 0
- Installing and removing 0
- Version requirements 0
- Running a program 0
- Building a stand-alone program 0
- How the merge works 0

Chapter 4. Creating and Publishing a Package 0

- What a package is 0
- Writing the routines 0
- Building the archive 0
- Publishing 0
- Packages for NetRexx and cREXX 0
- Trying a package before you publish it 0
- Common errors 0

Chapter 5. Accounts, Tokens and Owners 0

- Getting an account 0
- Tokens 0
- Owners 0

Chapter 6. The Web Interface 0

- What you can do in a browser 0
- What needs the client 0
- Requesting an account 0
- Safety of the pages 0

Chapter 7. Installing the Server 0

- Packages 0
- Directories 0
- SELinux 0
- Apache 0
- Firewall 0
- HTTPS 0
- Sign-up mail (optional) 0
- The first account 0

Chapter 8. Operating the Server 0

- Administration commands 0
- Where things are stored 0
- Backup and restore 0
- Upgrading 0
- Limits 0
- Troubleshooting 0

Chapter 9. Protocol Reference 0

Response format.... 0
Actions without a login.... 0
Actions that need a token.... 0
Error codes.... 0

Chapter 10. Security.... 0
Tokens and invitations.... 0
Publishing.... 0
Sign-up requests.... 0
The web pages.... 0
What RXPIP does not protect against.... 0

Chapter 11. Restrictions and Known Problems.... 0
What has been tested.... 0
What has not been tested.... 0
Known limitations.... 0

Appendix A. Command Summary.... 0
Client commands.... 0

Appendix B. Server Administration Commands.... 0

Appendix C. Environment Variables and Files.... 0
Environment variables.... 0
Files.... 0

Appendix D. Contents of the Kit.... 0

Preface

This manual introduces RXPIP, a package manager for the REXX language. It tells you how to install the `rxpip` client on Linux, macOS and Windows, how to find, install and use packages, how to write and publish your own, and how to install and run the repository server that stores them.

It is written for the REXX programmer who wants to reuse other people's routines without copying source files around, and for the person who runs a package repository such as www.rexxpository.org. No knowledge of other package managers is assumed, but readers who know `pip` will recognise the commands.

How this manual is organized

- **Chapter 1, Introduction**, says what RXPIP is, how its parts fit together and how a program uses a package.
- **Chapter 2, Installing the Client**, installs `rxpip` on Linux, macOS and Windows and points it at a repository.
- **Chapter 3, Using Packages**, covers searching, installing, running and building programs that need packages.
- **Chapter 4, Creating and Publishing a Package**, shows how to write a package, build the archive and publish it.
- **Chapter 5, Accounts, Tokens and Owners**, explains who may publish what.
- **Chapter 6, The Web Interface**, describes what can be done with an ordinary web browser, and what cannot.
- **Chapter 7, Installing the Server**, installs the repository server on AlmaLinux 9 with Apache.
- **Chapter 8, Operating the Server**, covers accounts, invitations, backups and troubleshooting.
- **Chapter 9, Protocol Reference**, defines the HTTP interface for people who write other clients.
- **Chapter 10, Security**, describes how tokens, uploads and archives are protected.
- **Chapter 11, Restrictions and Known Problems**, lists what does not work yet and what has not been tested.
- **Appendix A** summarizes the client commands, **Appendix B** the server administration commands, **Appendix C** the environment variables and files, and **Appendix D** the contents of the kit.

Conventions

Commands and file names are shown in `monospaced type`. In command descriptions, words in angle brackets such as `<name>` are values you supply, and square brackets mean that the item is optional. Examples of terminal sessions are shown in framed figures; lines you type are not marked separately, because the context makes clear which lines you type.

Client means the `rxpip` program on your own computer. *Server* or *repository* means the CGI program and the directory of packages behind a web address such as https://www.rexxpository.org/. *Package* means a named, versioned set of REXX routines that can be installed.

Summary of changes

This edition is kept up to date as RXPIP grows; each change is listed here, newest first.

- **12 October 2026 (second edition).** Clients for ooRexx, NetRexx and cREXX in addition to Regina (artifact platforms `any`, `netrexx`, `crexx`); moderated sign-up through the web interface, with an approval mail to the administrator; a test matrix that runs one scenario against every client.
- **12 October 2026 (first edition).** First edition. Covers the server and client as they stand after the hardening work of early October: header-only tokens, invitation-only accounts, per-package owner lists, upload limits, archive checks, the read-only web interface, and `>=` version requirements in `REQUIRE` lines.

Chapter 1. Introduction

RXPIP is a package manager for REXX in the spirit of `pip` for Python. A package is a small archive of REXX source, a name and a version. A repository stores packages and serves them over HTTP or HTTPS. The `rxpip` client, itself a REXX program, talks to the repository, installs packages on your computer and builds programs that use them.

The parts of RXPIP

Part	What it is	Written in
Client	The `rxpip` command: search, install, run, build, publish, log in. One client per REXX family; all behave the same	Regina REXX / ooRexx, NetRexx, cREXX
Server	A CGI program run by Apache; stores packages, accounts and owners as plain files	Regina REXX
Web interface	Package index, search, package pages and the account-request form served by the same CGI program	Regina REXX (HTML)
Archive	A package file: a gzip-compressed tar of `.rexx` source files, extension `.rxpkg`	—

Figure 1. The parts of RXPIP

There is deliberately only one server implementation. The repository does not care which REXX interpreter will eventually run a package; it stores what the publisher gives it and labels it (see "Artifacts" below).

How a program uses a package

A program declares what it needs in a comment, one line per package:

```
/* demo.rexx */
/* REQUIRE mathutils */
/* REQUIRE greetutils>=1.0.0 */

say Shout('world')
say 'square of 7 is' Square(7)
```

Figure 2. A program with REQUIRE lines

`rxpip run demo.rexx` reads the `REQUIRE` lines, installs anything that is missing together with its own dependencies, and then runs the program. `rxpip build demo.rexx` does the same but writes out a single self-contained REXX file instead of running it.

Why run and build, and not just install

Classic REXX interpreters, Regina included, do not look in the current directory for a file named after an external routine. A file called `Square.rexx` is not found when your program calls `Square(7)`. RXPIP therefore uses the one method that works identically everywhere: it joins your program, a safety `exit`, and the source of every installed package into one ordinary REXX program, in which every required routine is simply another label. `rxpip install` on its own only fetches the files; `run` and `build` make them callable.

Artifacts: source and bytecode

Each version of a package can carry several *artifacts*. An artifact has a *kind* and a *platform*:

Kind	Platform	Meaning
`SOURCE`	`any`	Classic REXX source. The default; installed by the Regina and ooRexx clients.
`SOURCE`	`netrexx`	NetRexx source (`.nrx`, one class per file); installed by the NetRexx client.

Kind	Platform	Meaning
`SOURCE`	`crexx`	cREXX Level B source (`.crexx` modules); installed by the cREXX client.
`BYTECODE`	for example `crexx-linux-x86_64`	A precompiled form for one interpreter, operating system and architecture. Optional; published in addition to the source artifact, never instead of it.

Figure 3. Artifact kinds

When installing, the client asks the server which artifacts exist for the version, takes an exact match for its own platform tag if there is one, and otherwise falls back to the source artifact for its own language (`any`, `netrexx` or `crexx`). A client never installs another language's artifact. In this release only source artifacts are in use; the bytecode kind is reserved for later (see Chapter 11).

Supported REXX interpreters

The server is written for Regina REXX 3.9 and serves every client. There is one client for each family of REXX:

Interpreter	Client	Installs	What `rxpip run` does
Regina 3.9	`client/rxpip.rexx`	`SOURCE/any`	merges program and packages into one REXX file and runs it
ooRexx 5.x	`client/oorexx/rxpip-oorexx`	`SOURCE/any` (same packages as Regina)	the same merge, started through a small runner
NetRexx 4.x	`client/netrexx/rxpip-netrexx`	`SOURCE/netrexx`	compiles program and package classes with NetRexxC, runs the class
cREXX 1.0 beta 3	`client/crexx/rxpip-crexx`	`SOURCE/crexx`	compiles with `rxc`, assembles with `rxas`, runs with `rxvm`

Figure 4. The clients

All clients use the same commands, print the same messages and share the settings and the login in the home directory. Where this manual shows `rxpip`, use the launcher for your interpreter.

Chapter 2. Installing the Client

What you need

- **Regina REXX** 3.9 or later, with `regina -a` working. The `-a` option makes every command-line argument arrive separately; without it, an argument with spaces such as `--summary="a short text"` would be split apart.
- **curl**, which carries every request. No REXX HTTPS support is needed.
- **tar** and **gzip**, which unpack and create packages.
- On Windows, the same three; Windows 10, Windows 11 and Windows Server 2019 or later include `curl` and `tar`.

Linux and macOS

Copy the client into a directory on your `PATH` under the name `rxpip`, and make it executable:

```
$ sudo install -m 755 client/rxpip.rexx /usr/local/bin/rxpip
$ rxpip help
```

Figure 5. Installing the client on Linux or macOS

The first line of the client is `#!/usr/bin/regina -a`, so Regina must be at `/usr/bin/regina`. If it is elsewhere, link it there, or create a small wrapper script named `rxpip` that contains `exec regina -a /path/to/rxpip.rexx "$@"`.

Windows

1. Install Regina for Windows and check that `regina -v` works in a command prompt.
2. Check that `curl --version` and `tar --version` work.
3. Put `rxpip.rexx` and `rxpip.bat` together in one folder, for example `C:\rxpip`.
4. Add that folder to your `PATH`, open a new command prompt and run `rxpip help`.

`rxpip.bat` contains a single command, `regina -a "%~dp0rxpip.rexx" %*`, which finds the REXX file next to itself.

The other clients

ooRexx. Needs ooRexx 5 (`rexx`) and the same curl and tar. Use the launcher `client/oorexx/rxpip-oorexx` (Linux, macOS) or `rxpip-oorexx.bat` (Windows); it passes the arguments in a form the shared client can split again, because ooRexx hands over the command line as one string. The classic packages you install are the same ones the Regina client uses.

NetRexx. Needs Java 8 or later and `NetRexxC.jar` from the NetRexx distribution. Set `NETREXX_JAR` to its path (or put the jar next to the launcher) and run `client/netrexx/rxpip-netrexx` or `rxpip-netrexx.bat`. The launcher compiles `Rxpip.nrx` the first time it runs.

cREXX. Needs cREXX 1.0 beta 3 or later: `rxc`, `rxas` and `rxvm` on the `PATH`, or `CREXX_HOME` set to the installation. Run `client/crexx/rxpip-crexx` or `rxpip-crexx.bat`; the client is compiled on first use.

```
$ export NETREXX_JAR=$HOME/netrexx/lib/NetRexxC.jar
$ client/netrexx/rxpip-netrexx install nxutils
$ client/crexx/rxpip-crexx install cxutils
$ client/oorexx/rxpip-oorexx list
```

Figure 6. Using the other clients

Each language keeps its installed packages apart: Regina and ooRexx use `lib/`, `pkgs/` and `installed.txt` in the home directory; NetRexx uses the `netrexx/` subdirectory and cREXX the `crexx/` subdirectory with the same layout. `config.txt`, `credentials.txt` and `cache/` are shared.

Pointing the client at a repository

The client keeps its settings and its installed packages in a directory called the *home*: `~/.rxpip` on Linux and macOS, `%USERPROFILE%\.rxpip` on Windows, or the directory named by the environment variable `RXPIP_HOME`. The first run creates it with default settings:

```
$ rxpip config
REPO_URL: https://www.rexxpository.org/cgi-bin/rxpip_server.rexx
LIB_DIR: /home/anna/.rxpip/lib
INTERPRETER: regina
```

Figure 7. Default client settings

To use another repository, set `REPO_URL`:

```
$ rxpip config set REPO_URL https://repo.example.org/cgi-bin/rxpip_server.rexx
```

`rxpip config get <key>` shows one setting. `INTERPRETER` names the program that `rxpip run` starts; it must accept Regina's `-a` option.

Files in the home directory

Item	Contents
`config.txt`	The settings `REPO_URL`, `LIB_DIR` and `INTERPRETER`
`installed.txt`	One record per installed package: name, version, author, summary, requirements, files
`lib/`	The `.rexx` files of all installed packages, flattened into one directory
`pkgs/<name>/<version>/`	The unpacked archive of each installed package
`cache/`	Downloaded `.rxpkg` files, kept for reuse
`credentials.txt`	Your saved login for each repository (created by `rxpip register` or `rxpip login`; mode 600 on Linux and macOS)

Figure 8. Contents of the client home directory

> **Note:** `credentials.txt` holds a secret. Do not put it in version control or share it. On Windows the file cannot be restricted by mode and relies on the permissions of your user profile.

Chapter 3. Using Packages

Finding packages

```
$ rxpip search math

NAME: mathutils
VERSION: 1.0.0
AUTHOR: Anna Example
SUMMARY: Square, cube and primality helpers
REQUIRES:

$ rxpip show greetutils

NAME: greetutils
VERSION: 1.0.0
AUTHOR: Anna Example
SUMMARY: Greeting helpers
REQUIRES: strutils>=1.0.0
```

Figure 9. Searching and showing a package

`search` looks for the term in package names and summaries (without regard to case) and lists the latest version of each match. `search` with no term lists everything. `show <name>` prints the details of one package. You can do the same in a web browser; see Chapter 6.

Installing and removing

Command	Effect
`rxpip install <name>`	Install the latest version of a package and everything it requires
`rxpip install <name>==<version>`	Install the stated version
`rxpip uninstall <name>`	Remove a package from `lib/` and from the installed list. Packages it required are left in place.
`rxpip list`	List installed packages with their versions and summaries
`rxpip freeze`	Print `name==version` for every installed package, one per line, suitable for a requirements file

Figure 10. Installing and removing packages

Installing a package that is already installed does nothing except to check that its requirements are still present. Installation downloads the archive into `cache/`, unpacks it into `pkgs/` and copies its `.rexx` files into `lib/`.

Version requirements

A package or a `REQUIRE` line can state a minimum version with `>=`:

```
/* REQUIRE strutils>=1.0.0 */
```

The client installs the **latest** version it can find and checks that it is at least the minimum; if the repository's newest version is older, it says so and installs nothing. If a sufficient version is already installed, nothing is fetched. `==` pins an exact version on the command line (`rxpip install greetutils==1.0.0`).

Running a program

```
$ rxpip run demo.rexx first second
rxpip: installing mathutils 1.0.0 (SOURCE/any) ...
rxpip: installing greetutils 1.0.0 (SOURCE/any) ...
rxpip: installing strutils 1.0.0 (SOURCE/any) ...
HELLO, WORLD!!!
square of 7 is 49
cube of 3 is 27
17 is prime
```

Figure 11. Running a program that requires packages

`rxpip run <file.rexx> [arguments]` scans the file for `REQUIRE` lines, installs what is missing, builds the merged program in a temporary file, runs it with the interpreter named in `INTERPRETER`, deletes the temporary file, and exits with the program's own return code. Arguments after the file name are passed to your program unchanged, spaces included.

A `REQUIRE` directive must be a comment that begins its line: `/* REQUIRE name */` or `/* REQUIRE name>=version */`. Directives elsewhere are ignored.

Building a stand-alone program

```
$ rxpip build demo.rexx --output=demo_standalone.rexx
rxpip: mathutils 1.0.0 already installed
rxpip: greetutils 1.0.0 already installed
rxpip: strutils 1.0.0 already installed
rxpip: wrote self-contained program to demo_standalone.rexx
$ regina demo_standalone.rexx
HELLO, WORLD!!!
...
```

Figure 12. Building a stand-alone file

`rxpip build <file.rexx> [--output=<file>]` writes the merged program to the output file (default `build_<file>`). The result needs no RXPIP and no network: give it to someone who has only Regina. Use `rxpip scan <file.rexx>` to install the required packages without running or building anything.

How the merge works

The merged program consists of three parts in this order: your source; a line `exit`, so that execution can never run on into the library code; and the source of every installed package file. Because every package routine becomes an internal label of one program, two packages must not define a routine with the same name, and a package routine can call any other.

Chapter 4. Creating and Publishing a Package

What a package is

A package is a gzip-compressed tar archive, conventionally named `<name>-<version>.rxpkg`, that contains one or more `.rexx` files at its top level. The name, version, author, summary and requirements are not inside the archive; you give them when you publish.

Writing the routines

Write your routines as labelled procedures, as in the example `mathutils.rexx` of the kit:

```
/* mathutils.rexx -- part of the "mathutils" package
   Exposes:  Square(n), Cube(n)
*/

Square: procedure
  parse arg n
  return n * n

Cube: procedure
  parse arg n
  return n * n * n
```

Figure 13. A package source file

Follow these rules:

- Do not put executable statements at the top of the file. Package source is appended after your caller's `exit`, and the first routine label must be reached only by a call.
- Declare every routine with `procedure`, so its variables stay private. Use `expose` only for variables you really intend to share.
- Do not give a routine the name of a REXX built-in function such as `Reverse` or `Length`. Regina resolves labels before built-ins, and a routine that calls the built-in of its own name calls itself without end.
- Choose routine names that are unlikely to collide with other packages, because all routines end up in one program.
- Note in a comment which other packages you rely on, and list them with `--requires` when you publish.

Building the archive

```
$ tar czf mathutils-1.0.0.rxpkg mathutils.rexx
$ tar tzf mathutils-1.0.0.rxpkg
mathutils.rexx
```

Figure 14. Creating a package archive

The same command works in a Windows command prompt. The server accepts an archive only if all of these hold:

- it is a gzip-compressed tar archive, at most 5 MiB (5 242 880 bytes) in size unless the operator has changed the limit;
- it holds at most 200 members, all regular files at the top level: no directories, symbolic or hard links, or special files;
- every file name is 1 to 100 characters from letters, digits, `.`, `_` and `-`, and does not begin with a dot.

Publishing

Publishing needs an account (Chapter 5). Once you are logged in:

```
$ rxpip publish mathutils-1.0.0.rxpkg --name=mathutils --version=1.0.0 \
    --author="Anna Example" --summary="Square, cube and primality helpers" \
    --requires="strutils>=1.0.0"
STATUS: OK
NAME: mathutils
VERSION: 1.0.0
KIND: SOURCE
PLATFORM: any
MESSAGE: published
```

Figure 15. Publishing a package

Option	Meaning
`--name=<name>`	Package name: 1 to 64 characters from letters, digits, `.`, _ and `-`; may not begin with a dot or contain `..`. Names are case-sensitive.
`--version=<version>`	Dotted numbers such as `1.0.0` or `2.3`. Required.
`--author=<text>`	Optional; at most 100 characters, no control characters.
`--summary=<text>`	Optional one-line description; at most 200 characters. It is what `search` looks at.
`--requires=<list>`	Optional; names, or `name>=version`, separated by spaces or commas; at most 500 characters in all.
`--kind=SOURCE|BYTECODE`	Default `SOURCE`.
`--platform=<tag>`	Default `any`. For bytecode, a tag such as `crexx-linux-x86_64`.
`--token=<token>`	Use this token instead of the saved login (see Chapter 5). The environment variable `RXPIP_TOKEN` does the same without putting the token in your shell history.

Figure 16. Options of rxpip publish

Rules of publishing

- **The first publication of a name claims it.** You become its owner and only owners can publish new versions.
- **Releases are immutable.** Publishing a version, kind and platform that already exists is refused with a conflict. Publish a new version instead.
- **Metadata is set once per version** by its first artifact. If a later artifact of the same version (for example a bytecode artifact) supplies different metadata, the server keeps the original and says so in a `NOTE:` line.
- A refused upload leaves nothing behind and does not claim the name.

Packages for NetRexx and cREXX

A NetRexx package is a tar of `.nrx` files, each defining one class; a cREXX package is a tar of `.crexx` files, each a module with a `namespace` line. Publish them with the platform that names the language, so that only the matching client installs them:

```
$ tar czf nxutils.rxpkg NxUtils.nrx
$ rxpip-netrexx publish nxutils.rxpkg --name=nxutils --version=1.0.0 --platform=netrexx
$ tar czf cxutils.rxpkg cxutils.crexx
$ rxpip-crexx publish cxutils.rxpkg --name=cxutils --version=1.0.0 --platform=crexx
```

Figure 17. Publishing NetRexx and cREXX packages

In a program, name the packages in a comment on a line of its own, as for classic REXX: `/* REQUIRE nxutils */` (NetRexx) or `-- REQUIRE cxutils` (cREXX; a `/* ... */` comment works too). `rxpip run prog.nrx`

or `rxpip run prog.crexx` then installs what is missing, compiles everything together and runs it. The package name may carry artifacts for several languages at once.

Trying a package before you publish it

You can test a package without a repository: put the file in `~/.rxpip/lib/` yourself and use `rxpip run` or `rxpip build`, or write the `REQUIRE` lines and merge by hand. For a real trial, install your own test server (Chapter 7) or use a test package name on the public repository.

Common errors

Message code	Cause and remedy
`NO_TOKEN`, `BAD_TOKEN`	You are not logged in, or the token is wrong or revoked. Run `rxpip login` or `rxpip register`.
`FORBIDDEN`	The name belongs to someone else. Ask an owner to add you (`rxpip owner add`), or choose another name.
`ALREADY_EXISTS`	That version has been published. Increase the version.
`BAD_ARCHIVE`	The archive failed a check; the message says which (link, directory, nested path, bad name, not a gzip tar).
`TOO_LARGE`	The archive is over the upload limit.
`BAD_NAME`, `BAD_VERSION`, `BAD_METADATA`, `BAD_REQUIRES`	A value has characters or a length the server does not accept.

Figure 18. Frequent publishing errors

Chapter 5. Accounts, Tokens and Owners

Anyone can search, show, install and download. Publishing, and changing who owns a package, requires an account. Accounts are **approved by the repository administrator**: every user who can publish is somebody whose code other people will run, and an open sign-up on a code repository is an easy way to distribute malware.

Getting an account

There are two ways to get one. **Ask on the web page:** open `<repository address>?action=SIGNUP`, enter the name you want, your e-mail address and your location, and send the form. The administrator receives a mail, checks the request and approves or rejects it. On approval you receive a mail with a single-use invitation code at the address you gave. **Or** the operator creates an invitation and gives it to you directly (Chapter 8). Either way you redeem the code once:

```
$ rxpip register --user=anna --invite=inv_1a2b3c4d_0123456789abcdef0123456789abcdef
registered as anna; your token was saved in /home/anna/.rxpip/credentials.txt
Treat that file like a password. You can now: rxpip publish ...
$ rxpip whoami
STATUS: OK
USER: anna
```

Figure 19. Registering

User names are 3 to 32 characters from lowercase letters, digits, _ and `-`, and may not begin with `-`. The server creates the account and a first token and shows the token exactly once; the client saves it for you. The invitation can be passed in the environment variable `RXPIP_INVITE` instead of the option.

Tokens

A token proves who you are. It looks like `rxp_1a2b3c4d_` followed by 48 hexadecimal digits, and is generated by the server. The server stores only a salted hash of it, so a lost token cannot be recovered, only replaced.

Command	Effect
`rxpip login --token=<token>`	Check the token with the server and save it as your login. The token can come from `RXPIP_TOKEN` instead.
`rxpip whoami`	Show the user name belonging to your saved token
`rxpip logout`	Forget the saved login on this computer (the token itself stays valid)
`rxpip token new [--label=<text>]`	Create another token, for example for a build machine. Shown once. At most 20 tokens per user.
`rxpip token list`	List your tokens: id, label and creation date. Never shows the secrets.
`rxpip token revoke <id>`	Invalidate one of your tokens, using the 8-digit id from the list

Figure 20. Token commands

> **Note:** A token is accepted only in the `X-Rxpip-Token` request header. A token placed in a URL is refused, because URLs are written to server logs. The client never puts a token in a URL or on a command line; it passes it to `curl` in a private temporary file.

Owners

Every package has a list of owners. The first publisher is the first owner. An owner may publish new versions and may add or remove owners:

```
$ rxpip owner list mathutils
STATUS: OK
NAME: mathutils
OWNERS: anna
$ rxpip owner add mathutils bob
STATUS: OK
NAME: mathutils
OWNERS: anna bob
$ rxpip owner remove mathutils anna
STATUS: OK
NAME: mathutils
OWNERS: bob
```

Figure 21. Managing owners

```
$ rxpip token new --label=ci
STATUS: OK
USER: anna
TOKEN: rxp_8b1dcce2_12b9af64a7b3b978155de7fa7e99d9b86585da0963522ff4
MESSAGE: new token created. Store it now; it is shown only once.
$ rxpip token list
STATUS: OK
COUNT: 2

TOKEN_ID: 718d5f4c
LABEL: registration
CREATED: 20261012 21:09:17

TOKEN_ID: 8b1dcce2
LABEL: ci
CREATED: 20261012 21:09:19
```

Figure 22. Creating and listing tokens (example values)

The user to be added must already have an account. The last owner of a package cannot be removed. If every owner loses access, the operator can restore the list from the server (Chapter 8).

Chapter 6. The Web Interface

The repository has a web interface for browsing and for requesting an account. Open the repository's address, for example https://www.rexxpository.org/, in any web browser. It is plain HTML: it needs no JavaScript and sets no cookies.

What you can do in a browser

Page	Address	What it shows
Package index	`/` (the CGI program with no action)	All packages with latest version, summary and owners, and a search box
Search	`?q=<term>`	The index limited to names and summaries containing the term
Package page	`?action=PKG&name=<name>`	Summary, author, owners, requirements (linked), the install command, the files of the version with download links, and a list of all versions
Older version	`?action=PKG&name=<name>&version=<version>`	The same page for one version

Figure 23. Pages of the web interface

Each artifact on a package page has a **Download** link, so a package can be fetched without `rxpip`. The download is the `.rxpkg` archive itself; unpack it with `tar xzf` and copy the `.rexx` files where you want them.

What needs the client

A browser cannot install packages into your REXX environment, run a program, or hold a login. These remain tasks for `rxpip`:

- installing, uninstalling, running and building (`rxpip install`, `run`, `build`);
- registering with an invitation, logging in and managing tokens;
- publishing and managing owners.

There is no web form for uploading or owner changes in this release; see Chapter 11. Each package page shows the exact `rxpip install` command to copy.

Requesting an account

When the operator has enabled it (Chapter 7), the page `?action=SIGNUP` shows a form with three fields: the name you want to log in with, your e-mail address and your location. Names follow the rules of Chapter 5. Sending the form stores a request and mails the administrator; you see a confirmation, nothing more happens until the administrator decides. The administrator opens the link in the mail, sees your data and presses **Approve** or **Reject**. Opening the link alone changes nothing. On approval an invitation code is mailed to you. A request expires after 14 days; the server keeps at most 100 pending requests and 3 per network address.

Safety of the pages

Every value that originates from package metadata (names, summaries, authors) is escaped before it is placed in a page, and the server sends a content security policy that forbids scripts and any loading from other sites. A package author therefore cannot put active content on the repository's pages.

Chapter 7. Installing the Server

This chapter installs the repository on AlmaLinux 9 (AlmaLinux 8 is the same) with Apache `httpd`, as used for www.rexxpository.org on a Hetzner cloud server. You need root access, a domain name pointing at the server, and Regina REXX.

> **Note:** The commands in this chapter follow the standard procedure for AlmaLinux, but the author's test system was not AlmaLinux: the server program is tested against an Apache-like test harness only. Expect to check the SELinux and Firewall sections of this chapter first if something does not work.

Packages

```
# dnf install -y epel-release
# dnf install -y httpd mod_ssl curl tar gzip coreutils regina-rexx
# regina -v
```

Figure 24. Installing the prerequisites

If `regina-rexx` is not available, build Regina from its source distribution (`./configure && make && make install`) and make sure `/usr/bin/regina` exists, for example with `ln -s /usr/local/bin/regina /usr/bin/regina`. The server uses only `tar`, `sha256sum`, `head`, `od`, `ls`, `mv`, `rm` and `mkdir` and reads `/dev/urandom`.

Directories

```
# mkdir -p /var/www/rexxpository/{cgi-bin,repo,html}
# cp server/rxpip_server.rexx /var/www/rexxpository/cgi-bin/
# chmod 755 /var/www/rexxpository/cgi-bin/rxpip_server.rexx
# chown -R apache:apache /var/www/rexxpository/repo
# chmod 750 /var/www/rexxpository/repo
```

Figure 25. Laying out the server

All data the server writes lives in `repo/`, outside Apache's `DocumentRoot` (`html/`, which stays empty), so account data can never be served as a static file. The directory must be writable by the account Apache runs CGI programs as, `apache` on AlmaLinux. `repo/` will hold the package directories and two hidden directories, `.auth/` (users, token hashes, invitations, owner lists) and `.staging/` (uploads in progress).

SELinux

Leave SELinux enforcing. Give the CGI directory and the data directory the labels Apache expects:

```
# dnf install -y policycoreutils-python-utils
# semanage fcontext -a -t httpd_sys_script_exec_t "/var/www/rexxpository/cgi-bin(/.*)?"
# semanage fcontext -a -t httpd_sys_rw_content_t  "/var/www/rexxpository/repo(/.*)?"
# restorecon -Rv /var/www/rexxpository
```

Figure 26. SELinux labels

`ls -dZ` on the two directories should now show `httpd_sys_script_exec_t` and `httpd_sys_rw_content_t`. If something is denied, read `ausearch -m avc -ts recent` rather than disabling SELinux.

Apache

Create `/etc/httpd/conf.d/rexxpository.conf`. Start with the port 80 host; the certificate step adds HTTPS.

```
<VirtualHost *:80>
    ServerName www.rexxpository.org
    DocumentRoot /var/www/rexxpository/html

    ScriptAlias /cgi-bin/ /var/www/rexxpository/cgi-bin/
    <Directory "/var/www/rexxpository/cgi-bin">
        Options +ExecCGI
        AddHandler cgi-script .rexx
        Require all granted
    </Directory>

    RedirectMatch 302 "^/$" "/cgi-bin/rxpip_server.rexx"
    SetEnv RXPIP_REPO_ROOT /var/www/rexxpository/repo
    SetEnv RXPIP_MAX_UPLOAD 5242880
    LimitRequestBody 5300000
</VirtualHost>
```

Figure 27. The Apache virtual host

`RXPIP_REPO_ROOT` tells the program where its data is. `RXPIP_MAX_UPLOAD` is the largest archive in bytes; `LimitRequestBody` is Apache's own limit, set slightly higher so that the program can answer with its own clear message. Then:

```
# apachectl configtest
# systemctl enable --now httpd
$ curl -s "http://localhost/cgi-bin/rxpip_server.rexx?action=PING"
STATUS: OK
SERVER: RXPIP/1.0 (Regina REXX-Regina_3.9.5(MT) 5.00 25 Jun 2022)
```

Figure 28. Starting Apache and testing

Firewall

```
# firewall-cmd --permanent --add-service=http --add-service=https
# firewall-cmd --reload
```

A Hetzner cloud server also has a separate cloud firewall in the Hetzner console. Allow TCP ports 80 and 443 there, and port 22 from your own address. If the repository answers on the server but not from outside, look there first.

HTTPS

Tokens travel in request headers, so the repository must be served over HTTPS only. Point the DNS records of the host name at the server, then:

```
# dnf install -y certbot python3-certbot-apache
# certbot --apache -d www.rexxpository.org
# systemctl list-timers | grep certbot
```

Figure 29. Obtaining a certificate

Certbot adds the HTTPS host and redirects port 80 to it, and renews the certificate automatically.

Sign-up mail (optional)

Web sign-up is off until you give the server an administrator address. Install a mail transfer agent that provides `/usr/sbin/sendmail` (`dnf install postfix`, then `systemctl enable --now postfix`), allow Apache to send mail under SELinux (`setsebool -P httpd_can_sendmail 1`), and add three lines to the virtual host:

```
SetEnv RXPIP_ADMIN_EMAIL  you@example.org
SetEnv RXPIP_PUBLIC_URL   https://www.rexxpository.org/cgi-bin/rxpip_server.rexx
SetEnv RXPIP_MAIL_FROM    rxpip@rexxpository.org
```

Figure 30. Enabling sign-up requests

`RXPIP_ADMIN_EMAIL` receives the review links. `RXPIP_PUBLIC_URL` is the full address of the CGI program and is used to build those links, so that they never depend on a client-supplied host name. `RXPIP_MAIL_FROM` is the sender. Set up SPF and DKIM for the sender domain, or the mails may be filed as spam. `RXPIP_SENDMAIL` can name another command that reads a finished message on standard input.

The first account

Create your own account on the server (the command prints the token once, so copy it):

```
# sudo -u apache env RXPIP_REPO_ROOT=/var/www/rexxpository/repo \
    regina -a /var/www/rexxpository/cgi-bin/rxpip_server.rexx admin adduser anna
user anna created. token (shown once): rxp_1a2b3c4d_...
```

Figure 31. Creating the first user

Then, from any computer with the client: `rxpip config set REPO_URL https://www.rexxpository.org/cgi-bin/rxpip_server.rexx`, and `rxpip login --token=<token>`. Open the repository's address in a browser; the empty package index should appear.

Chapter 8. Operating the Server

Administration commands

Accounts are managed on the server by the operator, never over HTTP. Run the server program from the command line, as the web user so that file ownership stays correct. The program recognises command-line use because the CGI variable `GATEWAY_INTERFACE` is not set; a request arriving through Apache can never enter this mode.

```
# ADMIN='sudo -u apache env RXPIP_REPO_ROOT=/var/www/rexxpository/repo regina -a \
         /var/www/rexxpository/cgi-bin/rxpip_server.rexx admin'
# $ADMIN invite "for Anna"
invite code (single use, shown once): inv_1a2b3c4d_0123456789abcdef0123456789abcdef
redeem with: rxpip register --user=<name> --invite=inv_1a2b3c4d_...
```

Figure 32. Creating an invitation

Appendix B lists every administration command. In daily use you need `invite` to let someone in, `lock <user>` to cut off a user at once, and `setowners` to repair ownership when all owners have lost access.

Where things are stored

```
repo/<name>/<version>/PACKAGE.META
repo/<name>/<version>/ARTIFACTS.META
repo/<name>/<version>/artifacts/<KIND>-<PLATFORM>.rxpkg
repo/.auth/users/<user>.user
repo/.auth/tokens/<id>.tok          (salted hashes, not tokens)
repo/.auth/invites/<id>.inv[.used]
repo/.auth/owners/<package>.owners
repo/.staging/                      (uploads in progress)
```

Figure 33. Layout of the repository directory

Backup and restore

Everything is in plain files, so a copy of `repo/` is a complete backup; there is no database. Keep the backup readable by root only, because it contains the token hashes.

```
# tar czf /root/rexxpository-$(date +%F).tgz --exclude=repo/.staging \
    -C /var/www/rexxpository repo
```

Figure 34. Backing up

To restore, unpack into `/var/www/rexxpository/`, run `chown -R apache:apache` on `repo/`, and then `restorecon -R /var/www/rexxpository`.

Upgrading

Copy the new `rxpip_server.rexx` over the old one and keep its mode and SELinux label (`restorecon -v` on the file). Data files are not changed by an upgrade. Check with the `PING` action afterwards.

Limits

Setting	Default	Where
Largest upload	5 242 880 bytes	`SetEnv RXPIP_MAX_UPLOAD`; also Apache `LimitRequestBody`
Tokens per user	20	fixed

Setting	Default	Where
Members per archive	200	fixed
Name, summary, author, requirements	64, 200, 100, 500 characters	fixed

Figure 35. Server limits

Troubleshooting

Symptom	Likely cause and what to check
HTTP 500 and "Permission denied" in `/var/log/httpd/error_log`	SELinux label or file mode on `cgi-bin`: repeat the SELinux section of Chapter 7 and check `ls -Z`
`PING` works, publishing reports success but nothing is stored	`repo` is not writable by `apache` or lacks the `httpd_sys_rw_content_t` label
`ausearch -m avc -ts recent` lists `httpd_sys_script_t` denials	Read the denial and fix the label; `audit2why` explains it
"Premature end of script headers"	Regina not found by the CGI, or a program error. Run it by hand: `sudo -u apache env GATEWAY_INTERFACE=CGI/1.1 REQUEST_METHOD=GET QUERY_STRING=action=PING regina /var/www/rexxpository/cgi-bin/rxpip_server.rexx`
`SERVER_NO_HASH` or `SERVER_NO_RANDOM`	`sha256sum` or `/dev/urandom` is unavailable to the confined program; check `audit.log`
Works on the server, not from outside	`firewalld`, or the Hetzner cloud firewall
Large uploads fail with HTTP 413	Over `RXPIP_MAX_UPLOAD` or Apache's `LimitRequestBody`

Figure 36. Troubleshooting guide

Chapter 9. Protocol Reference

The client and the web interface use one HTTP interface, so other clients can be written in any language. Every request is addressed to the CGI program, with the action in the query string: `https://host/cgi-bin/rxpip_server.rexx?action=<ACTION>&...`.

Response format

Responses are plain text in lines of `KEY: value`. A response begins with `STATUS: OK` or `STATUS: ERROR`; an error adds `CODE:` and `MESSAGE:`. Lists have a `COUNT:` line, then records separated by blank lines.

```
STATUS: OK
COUNT: 1

NAME: mathutils
VERSION: 1.0.0
AUTHOR: Anna Example
SUMMARY: Square, cube and primality helpers
REQUIRES:
```

Figure 37. A list response

Errors of the actions that change something also carry a real HTTP status code (see the error table at the end of this chapter). `DOWNLOAD` returns the raw archive with `Content-Type: application/octet-stream`.

Actions without a login

Action	Method	Parameters	Result
(none)	GET	`q`	HTML package index
`PKG`	GET	`name`, `version`	HTML package page
`PING`	GET	—	Server identification
`LIST`	GET	—	Latest version of every package
`SEARCH`	GET	`q`	Packages whose name or summary contains `q`
`INFO`	GET	`name`, `version`	Metadata of one version (latest by default)
`VERSIONS`	GET	`name`	All versions, oldest first
`ARTIFACTS`	GET	`name`, `version`	Kind, platform and files of each artifact
`DOWNLOAD`	GET	`name`, `version`, `kind`, `platform`	The archive (defaults: latest, `SOURCE`, `any`)
`OWNERS`	GET	`name`	Space-separated owner list
`REGISTER`	POST	headers `X-Rxpip-Invite`, `X-Rxpip-User`	New user and first token

Figure 38. Actions that need no token

Actions that need a token

These are POST requests with the header `X-Rxpip-Token: <token>`. A token in the query string is refused.

Action	Headers	Result
`WHOAMI`	—	The user name
`NEWTOKEN`	`X-Rxpip-Label` (optional)	A new token, shown once
`TOKENS`	—	Id, label and creation date of your tokens

Action	Headers	Result
REVOKE	X-Rxpip-Token-Id	Revokes one of your tokens
ADDOWNER, REMOVEOWNER	X-Rxpip-Name, X-Rxpip-User	The resulting owner list
PUBLISH	X-Rxpip-Name, -Version, -Author, -Summary, -Requires, -Kind, -Platform; body: the archive bytes	Stores the artifact

Figure 39. Actions that need a token

For PUBLISH, the non-secret metadata may alternatively be given as query-string parameters of the same names, for the benefit of intermediaries that drop custom headers; headers win when both are present. The token has no such alternative.

Error codes

Code	HTTP	Meaning
BAD_NAME, BAD_VERSION, BAD_KIND, BAD_PLATFORM, BAD_ARTIFACT	400	A parameter is missing or not acceptable
BAD_METADATA, BAD_REQUIRES, BAD_LABEL, BAD_USER, BAD_TOKEN_ID	400	A value has a bad form, length or characters
TOKEN_IN_URL	400	A token was sent in the URL
EMPTY_BODY, SHORT_BODY, BAD_ARCHIVE	400	Missing, truncated or invalid upload
NO_TOKEN, BAD_TOKEN	401	No token, or unknown, revoked or malformed
FORBIDDEN, BAD_INVITE	403	Not an owner of the package; invitation not valid
NOT_FOUND, NO_SUCH_USER, NOT_AN_OWNER	404	The named thing does not exist
METHOD_NOT_ALLOWED	405	Wrong HTTP method for the action
ALREADY_EXISTS, USER_EXISTS, LAST_OWNER, TOO_MANY_TOKENS	409	The request conflicts with existing data
TOO_LARGE	413	Upload over the size limit
SERVER_NO_HASH, SERVER_NO_RANDOM	500	The server lacks SHA-256 or secure random numbers and refuses rather than weaken security

Figure 40. Error codes and HTTP status

Chapter 10. Security

A package repository distributes code that other people run, so its security matters more than its size suggests. This chapter says what RXPIP does and where its limits are.

Tokens and invitations

- Tokens and invitations are generated by the server from the operating system's random source. If the server cannot obtain secure random numbers or a SHA-256 program, it refuses to create accounts; it has no weaker fallback.
- Only a salted SHA-256 hash is stored. A token has 192 bits of randomness, so a single salted hash is adequate; a slow password hash would matter only for passwords chosen by people.
- The server accepts a token only in a request header. Secrets never appear in a URL, which would be logged, or in a command line, which other users of the machine can see with `ps`. The client hands the token to `curl` through a private temporary file and keeps its saved token in a file readable only by you.
- Responses that carry a new token are marked `Cache-Control: no-store`.
- Invitations are single use. Redeeming one is an atomic file rename, so two simultaneous attempts cannot both succeed.

Publishing

- The size limit is enforced before the body is read. The archive is checked before anything is kept: file types, names, count. Uploads are staged outside the package directories and moved into place in one step; a rejected upload leaves nothing and does not claim the name.
- Package names, versions and file names are checked against a strict list of characters before they are used in a path, which blocks path traversal.
- Free-text metadata may not contain control characters. A newline in a summary could otherwise inject fake fields into a stored record.
- Published releases are immutable and cannot be overwritten.

Sign-up requests

- A request changes nothing except one pending file. An account exists only after the administrator approves it, and the invitation goes to the address the applicant gave, which also shows that the applicant can read mail there.
- The review link carries a random 128-bit key; the server stores only its salted hash, the link expires after 14 days, and a request can be decided once. Opening the link (an HTTP GET) shows the request; approving or rejecting is a POST, so mail scanners and link previews cannot approve anything.
- Every field is validated before it is stored or mailed (name rules, a plain address syntax, no control characters), so nothing can inject extra mail headers. Field values are escaped in the pages. A hidden honeypot field, a global limit of 100 and a limit of 3 pending requests per network address slow down abuse.
- The links in the mail are built from `RXPIP_PUBLIC_URL`, never from the Host header the visitor sent.

The web pages

All metadata is escaped, and a content security policy forbids scripts and external resources.

What RXPIP does not protect against

- **A malicious package.** The repository checks the shape of an archive, not what the REXX code does. Treat a package as you would any program you download. Prefer packages whose source you have read, and use `rxpip build` to look at the merged program.
- **Name squatting and look-alike names.** The first publisher of a name owns it; there is no review or reserved-name list.
- **Floods of requests.** There is no rate limiting. For a public server, put `fail2ban` or a rate limit in front.
- **An unprotected connection.** Serve the repository over HTTPS only.

Chapter 11. Restrictions and Known Problems

What has been tested

The server and the Regina client, on Linux, are covered by an automated test suite of 180 checks, and a second suite runs one scenario of about 35 checks against each of the four clients (141 checks in all, with Regina, ooRexx, NetRexx and cREXX installed). All run against the real programs over HTTP, using a test harness that behaves like Apache (it forwards all request headers and runs the program as an ordinary user). The suite covers publishing, immutability, upload limits, archive attacks, invitations, tokens, owners, concurrent first publishes, the web pages, the client commands and the moderated sign-up (with a stand-in for sendmail that stores the mails).

What has not been tested

- **Windows.** The client and server contain Windows code paths (PowerShell for hashing and random numbers, `cmd.exe` commands for files). They were written with care but have never been run. On Windows the server cannot create a file atomically, so two simultaneous first publishes of one name, or two simultaneous uses of one invitation, are not race-safe. The installation checklist for Windows is in the repository file `DEPLOY.md`.
- **macOS.** Expected to work (it has `shasum` and the same shell tools) but not tried.
- **AlmaLinux with real Apache and SELinux.** The commands in Chapter 7 are the standard procedure but were not run on that system.
- **The other clients on Windows and macOS.** The ooRexx, NetRexx and cREXX clients and their `.bat` launchers were run on Linux only, with ooRexx 5.3, NetRexx 4 and cREXX 1.0 beta 3.
- **Real mail delivery.** The sign-up mails were tested with a stand-in for sendmail; delivery through Postfix, SPF/DKIM and spam filtering are for the operator to verify.
- **Bytecode.** No client builds or installs bytecode artifacts yet (cREXX `.rxbin` would be the first candidate).

Known limitations

- **Bytecode artifacts** are accepted and stored, and the client prefers an exact platform match, but no tool yet produces them.
- **Few web forms.** Requesting an account is possible in the browser; publishing, owner management and token handling are available only through the client.
- **No removal.** There is no command to delete or withdraw a published version; the operator must remove files by hand. Releases are meant to be immutable.
- **No rate limiting** and no limit on the number of packages per user.
- **Simultaneous owner changes** to the same package, in the same instant, are resolved by the last writer.
- **Names are case-sensitive** on Linux, so `Mathutils` and `mathutils` are different packages; on a Windows server they would collide. Use lower case.
- **`LIB_DIR`** appears among the settings but is not used yet; packages always go into `lib/` under the home directory.
- **Package routines share one namespace** after merging. Two packages that define the same routine name conflict, and the later definition is not reached.
- **`rxpip uninstall`** does not remove packages that the removed package required.
- **The `==` operator** accepts any installed version that is at least the stated one, and does not downgrade.

Appendix A. Command Summary

Client commands

Command	Description
`rxpip help`	Show the command list
`rxpip search [term]`	Search names and summaries
`rxpip show <name>`	Details of a package
`rxpip install <name>[==<ver>]`	Install a package and its requirements
`rxpip uninstall <name>`	Remove a package
`rxpip list`	List installed packages
`rxpip freeze`	Installed packages as `name==version`
`rxpip scan <file.rexx>`	Install the packages a program requires
`rxpip run <file.rexx> [args]`	Install what is required, merge and run
`rxpip build <file.rexx> [--output=<file>]`	Install what is required, write a stand-alone program
`rxpip publish <file.rxpkg> --name= --version= [...]`	Upload a package (Chapter 4)
`rxpip register --user=<name> --invite=<code>`	Create an account from an invitation
`rxpip login [--token=<token>]`	Save a token as your login
`rxpip logout`	Forget the saved login
`rxpip whoami`	Show the logged-in user
`rxpip token new [--label=<text>]`	Create a token
`rxpip token list`	List your tokens
`rxpip token revoke <id>`	Revoke a token
`rxpip owner list <pkg>`	Show the owners of a package
`rxpip owner add <pkg> <user>`	Add an owner
`rxpip owner remove <pkg> <user>`	Remove an owner
`rxpip config [get <key> | set <key> <value>]`	Show or change settings

Figure 41. Client commands

Appendix B. Server Administration Commands

Run on the server as the web user: `sudo -u apache env RXPIP_REPO_ROOT=<repo> regina -a rxpip_server.rexx admin <command>`. A command that creates a secret prints it once.

Command	Description
`invite [note]`	Create a single-use invitation code
`adduser <name>`	Create a user and print its first token
`newtoken <user> [label]`	Create another token for a user
`users`	List the users
`tokens <user>`	List a user's token ids, dates and labels
`revoke <token-id>`	Revoke one token
`lock <user>`	Revoke every token of a user
`owners <package>`	Show the owners of a package
`setowners <package> <user> [<user>...]`	Replace the owner list (account recovery)

Figure 42. Administration commands

Appendix C. Environment Variables and Files

Environment variables

Variable	Used by	Meaning
`RXPIP_HOME`	client	Home directory of the client (default `~/.rxpip`)
`RXPIP_TOKEN`	client	Token to use instead of the saved login
`RXPIP_INVITE`	client	Invitation code for `rxpip register`
`RXPIP_REPO_ROOT`	server	Directory holding the repository data
`RXPIP_MAX_UPLOAD`	server	Largest accepted upload in bytes (default 5242880)
`RXPIP_ADMIN_EMAIL`	server	Where sign-up review mails go; sign-up is off while unset
`RXPIP_PUBLIC_URL`	server	Full address of the CGI program, used in mailed links
`RXPIP_MAIL_FROM`	server	Sender address of the mails (default rxpip@<server name>)
`RXPIP_SENDMAIL`	server	Mail command (default `/usr/sbin/sendmail -t -i`)
`NETREXX_JAR`	NetRexx client	Path of `NetRexxC.jar`
`CREXX_HOME`	cREXX client	cREXX installation directory (its `bin` holds rxc, rxas, rxvm)
`RXPIP_INTERPRETER`	client	Interpreter started by `run`; set by the ooRexx launcher

Figure 43. Environment variables

Files

File	Where	Contents
`config.txt`	client home	Settings
`credentials.txt`	client home	Saved logins (a secret)
`installed.txt`	client home	Installed package records
`PACKAGE.META`	repo/<name>/<version>/	Name, version, author, summary, requirements
`ARTIFACTS.META`	repo/<name>/<version>/	Kind, platform and files of each artifact
`<KIND>-<PLATFORM>.rxpkg`	repo/<name>/<version>/artifacts/	The stored archive
`.auth/`	repo/	Users, token hashes, invitations, owner lists, pending sign-up requests (`requests/`)

Figure 44. Files used by RXPIP

Appendix D. Contents of the Kit

File	Purpose
`client/rxpip.rexx`	The client
`client/rxpip.bat`	Windows wrapper for the client
`client/oorexx/`	ooRexx launchers (`rxpip-oorexx`, `.bat`)
`client/netrexx/`	NetRexx client `Rxpip.nrx` and launchers
`client/crexx/`	cREXX client `rxpip.crexx` and launchers
`server/rxpip_server.rexx`	The server, the web interface and the administration program
`examples/`	Sample packages (`mathutils`, `strutils`, `greetutils`, `nxutils`, `cxutils`) and sample programs (`demo.rexx`, `echoargs.rexx`, `hello.nrx`, `hello.crexx`)
`tests/run_tests.sh`	The automated test suite (server and Regina client)
`tests/run_client_matrix.sh`	The same scenario run against every client installed on the machine
`tests/cgi_harness.py`	Test harness that behaves like Apache CGI
`README.md`, `ARCHITECTURE.md`, `DEPLOY.md`	Overview, design notes, deployment notes
`docs/`	This manual

Figure 45. Files in the kit